



SIMATS ENGINEERING

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Assessment Tool 1 – Prompt Engineering Challenge

Name: Hemanth.P

Reg. No.: 192472285

Course Code: CSA6502 **Course Title:** Generative AI and Large Language Models

Q16. Function-Calling / Tool-Use Prompt

Question: Write a prompt that instructs an LLM on exactly when to call a `get_weather(location)` function versus answering directly, including the precise JSON schema the model must output when it decides to call the function.

Prompt

You are an assistant with access to one function:

`get_weather(location: string) -> returns current weather conditions for the given location.`

Decide whether to call this function based on these rules:

- Call `get_weather` ONLY if the user is asking about current, today's, or real-time weather/temperature/conditions for a specific place.
- Do NOT call the function if the user asks about historical weather, general climate facts, weather trends, or anything not tied to a real-time lookup - answer such questions directly instead.
- Do NOT call the function if no specific location is given; instead, ask the user to clarify the location.

When you decide to call the function, output ONLY a JSON object in exactly this schema, with no extra text before or after it:

```
{
  "function_call": {
    "name": "get_weather",
    "arguments": {
      "location": "<string, the location extracted from the user's message>"
    }
  }
}
```

When you decide NOT to call the function, respond normally in plain text with your direct answer.

Simple Grok-LLM Command (for reference / execution)

```
curl https://api.x.ai/v1/chat/completions \
-H "Authorization: Bearer $GROK_API_KEY" \
-H "Content-Type: application/json" \
-d '{
```

```
"model": "grok-4",
"messages": [
  {"role": "user", "content": "<paste the prompt above>"},
  {"role": "user", "content": "What is the weather like in Chennai right now?"}
]
```

Sample Model Output (running the prompt)

```
{
  "function_call": {
    "name": "get_weather",
    "arguments": {
      "location": "Chennai"
    }
  }
}
```

(For contrast, if the user instead asked "Why is it usually hot in Chennai in summer?", the model would respond directly in plain text without producing any JSON, since that is a general climate question, not a real-time lookup.)

Design Rationale

- Spelling out explicit trigger conditions (real-time query + specific location) versus explicit non-trigger conditions (historical/general questions, missing location) stops the model from over-calling or under-calling the function.
- Requiring "no extra text before or after" the JSON when calling the function keeps the output machine-parseable for downstream code that executes the function call.
- Instructing the model to ask for clarification when the location is missing avoids it guessing a location or calling the function with an empty/invalid argument.

Q18. Reflexion (Iterative Self-Critique) Prompt

Question: Write a prompt that has the model produce an initial draft answer, critique its own draft against specific quality criteria, and then produce a revised final version — distinct from simple fact-checking, this focuses on iterative improvement of the output itself.

Prompt

You will answer the user's question in three labeled stages.

Stage 1 - Draft:

Write your first-attempt answer to the question below. Label this section "Draft:".

Stage 2 - Critique:

Critically evaluate your own Draft against these quality criteria:

- Clarity: Is the explanation easy to follow?
- Completeness: Does it fully address the question, with no gaps?
- Correctness: Are all facts, logic, and calculations accurate?
- Conciseness: Is there unnecessary repetition or filler?

For each criterion, point out specific weaknesses in the Draft (not generic praise). Label this section "Critique:".

Stage 3 - Revised Final Answer:

Rewrite the answer, directly fixing every weakness identified in the Critique. Label this section "Revised Final Answer:".

Do not skip any stage, and do not let the Revised Final Answer repeat flaws already flagged in the Critique.

Question: {{user_question}}

Simple Grok-LLM Command (for reference / execution)

```
curl https://api.x.ai/v1/chat/completions \
-H "Authorization: Bearer $GROK_API_KEY" \
-H "Content-Type: application/json" \
-d '{
  "model": "grok-4",
  "messages": [
    {"role": "user", "content": "<paste the prompt above, with user_question filled in:
Explain why the sky is blue.>"}
  ]
}'
```

Sample Model Output (running the prompt)

Draft:

The sky is blue because sunlight scatters in the atmosphere. Blue light scatters more than other colors, so we see a blue sky.

Critique:

- Clarity: "scatters more than other colors" is vague - it doesn't explain why blue scatters more.
- Completeness: Doesn't name the phenomenon (Rayleigh scattering) or mention why sunsets appear red/orange, which is a natural follow-up.
- Correctness: Technically accurate but incomplete - omits the role of wavelength.
- Conciseness: Fine, no unnecessary repetition.

Revised Final Answer:

Sunlight looks white but contains all colors of the visible spectrum, each with a different wavelength. As sunlight enters the atmosphere, it collides with gas molecules and scatters in many directions - a phenomenon called Rayleigh scattering. Because blue light has a shorter wavelength than red or yellow light, it scatters roughly four times more strongly. This scattered blue light reaches our eyes from all parts of the sky, making the sky appear blue. At sunrise and sunset, sunlight passes through more atmosphere, scattering away most of the blue light before it reaches us and leaving the longer red and orange wavelengths dominant - which is why skies turn red at those times.

Design Rationale

- Separating Draft, Critique, and Revised Final Answer into distinct labeled stages prevents the model from silently polishing its wording without actually reasoning about what was wrong.
- Naming concrete criteria (clarity, completeness, correctness, conciseness) forces specific, actionable criticism instead of vague self-praise like "this looks good."
- Explicitly requiring the revision to fix every flagged weakness closes the loop between critique and revision, which is the core mechanism that distinguishes Reflexion from a single-pass fact-check.